

SESSION - 08

Inheritance



Table of Contents

Inheritance

Exercise 1 7

Task - Investigate the contentpage sling:resourceSuperType property 4

Task- Investigate the Foundation Page Component 5

Task – Delete the contentpage.html script 7

INHERITANCE

Inheriting Foundation Components

Components can be given a hierarchical structure to implement the inheritance of script files, dialog boxes, and so on. Therefore, it is possible for a specific 'page' component (or any component) to inherit from a 'base' component. For example, allowing inheritance of a script file for a specific part of the page, for example, the `<head>` tag.

During this course, you will inherit a few of the foundation components such as:

- (Foundation) page component (Which we covered during previous session)
- Responsive grid component (also known as a paragraph system component) (Which we will see in future sessions)

Components within AEM are subject to three different hierarchies:

1. Resource Type Hierarchy

This is used to extend components using the property **sling:resourceSuperType**. This enables the component to inherit from a 'base' component. For example, a text component will inherit various attributes from the foundation text component, including:

Scripts (resolved by Sling)

Dialog boxes

Descriptions (including thumbnail images, icons, and so on)

It is important to note that a local copy or instance of a component element (for example, body.html) will take precedence over an inherited element.

2. Container Hierarchy

This is used to populate configuration settings to the child component and is most commonly used in a responsive grid scenario. For example, configuration settings for the edit bar buttons, control set layout (edit bars, rollovers, and so on), and dialog box layout (inline, floating, and so on) can be defined on the parent component and propagated to the child components. Configuration settings (related to edit functionality) in cq:editConfig and cq:childEditConfig are propagated. These topics are covered in later modules.

3. Include Hierarchy

This is imposed at runtime by the sequence of includes. It is typically used by the designer, which in turn acts as the base for various design aspects of the rendering—including layout. Example: Information, CSS information, the available components in a responsive grid, and so on.

Exercise

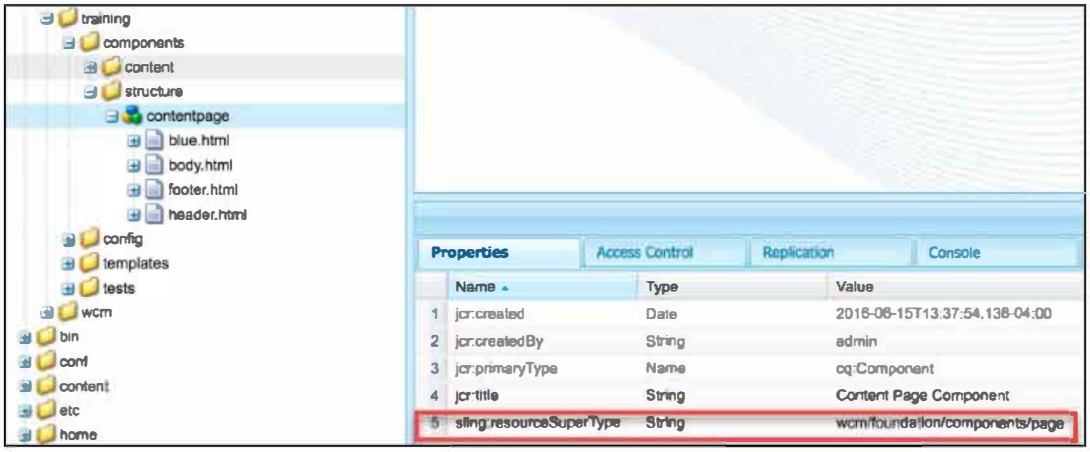
Review: When the /apps/training/components/structure/contentpage component was defined, we declared the Foundation Page component as the superType of our page-rendering component. By doing this, we allowed our contentpage component to initialize the WCM in order to take full advantage of the authoring environment. By declaring the Foundation Page component as its superType, the contentpage component will also inherit and make use properties, scripts, and other features of the Foundation Page component. Use of Foundation Page also ensures the Authoring UI is enabled.

Task – Investigate the contentpage sling:resourceSuperType property

1. **In CRXDE Lite, navigate to /apps/training/components/structure/contentpage.**

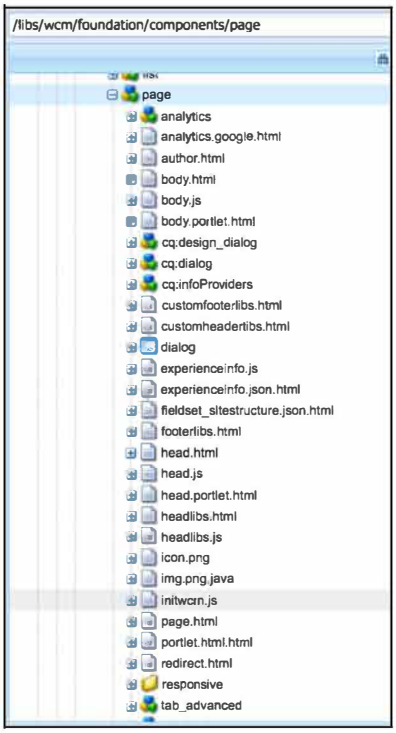
   Notice the sling:resourceSuperType property.

   Defining the sling:resourceSuperType property enables inheritance from a super type or, as it is sometimes called, a base type. Once you define the sling:resourceSuperType property with the value of wcm/foundation/components/page, the /apps/training/components/structure/contentpage component may now inherit scripts, properties, dialog box structures, and other elements from the Foundation Page component.



Task – Investigate the Foundation Page Component

1. **Using CRXDE Lite, navigate to /libs/wcm/foundation/components/page. You will note that the Foundation page component contains all the pieces necessary to be a complete page-rendering component.**



2. **Take particular notice of the default scripts: page.html and head.html:**

page.html

```
…
<head data-sly-include="head.html"></head>
    <body data-sly-use.body="body.js"
class="${body.cssClasses}"
            data-sly-include="body.html"></body>
…
```

head.html

```
…
<meta http-equiv="content-type" content="text/html; charset=UTF-8"
          data-sly-use.head="head.js"
          data-sly-use.headlibRenderer="headlibs.html"
          data-sly-
use.clientLib="/libs/granite/sightly/templates/clientlib.html"/>
    <meta data-sly-test.keywords="${head.keywords}" name="keywords"
content="${keywords}"/>
    <meta data-sly-test.description="${head.description}"
name="description" content="${description}"/>
    <sly data-sly-call="${headlibRenderer.headlibs @
designPath=head.designPath}" />
    <sly data-sly-include="author.html" />
    <sly data-sly-
include="/libs/cq/cloudserviceconfigs/components/servicelibs/servicelibs.j
sp" />
```

```
    <sly data-sly-include="customheaderlibs.html" />
    <sly data-sly-test.faviconpath="${head.faviconPath}">
    <link rel="icon" type="image/vnd.microsoft.icon"
href="${faviconpath}">
    <link rel="shortcut icon" type="image/vnd.microsoft.icon"
href="${faviconpath}">
    </sly>
    <title>${head.title}</title>
```

Task – Delete the contentpage.html script

1. **Using CRXDE Lite, delete /apps/training/components/structure/contentpage/contentpage.html script by right-clicking it and selecting Delete.**
2. **Save changes.**
3. **Using the Sites Console, open Sites > We.Train > English**

Notice that the page renders just fine, even without the default script, contentpage.html. The proper rendering of the page is possible because the contentpage component is inheriting several scripts from the Foundation Page component.

(Optional) Extra Credit Task – Build the rendering chain

Use CRXDE Lite to follow the Sling URL decomposition and resource resolution process.

Use CRXDE Lite to find the initial rendering scripts and build the rendering chain for the **Sites** > **We.Train** > **English** page.